Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4

По дисциплине: «Обработка изображений в интеллектуальных системах»
Тема: «Трекинг множественных объектов»

Выполнил:
Студент 4 курса
Группы ИИ-25
Трошин И.Д.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: исследовать применение алгоритмов трекинга на базе обученной сети-детектора объектов.

Общее задание

1. Используя сеть-детектор, обученный в ЛР 3, реализовать логику для отслеживания множественных объектов, используя библиотеку Ultralytics YOLO;
2. Применять алгоритмы BoT-Sort и ByteTrack (задействовать соответствующие конфигурационные файлы);
3. Исследовать изменения параметров в конфигурационных файлах и их влияние на качество трекинга;
4. В качестве исходных видеоматериалов для экспериментов использовать видеоролики из сети (например, из YouTube), содержащие множественные объекты классов из ЛР 3;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

Вариант 13

В-т	Детектор	Датасет
13	YOLO26n	**Люди:** https://universe.roboflow.com/leo-ueno/people-detection-o4rdr/dataset/10

Ход работы

Код программы

```python
from ultralytics import YOLO

model = YOLO("best.pt")

results = model.track(
    source="video.mp4",
    tracker="bytetrack.yaml",
    conf=0.1,
    save=True,
    stream=True
)

for result in results:
    pass
```

```python
from ultralytics import YOLO

model = YOLO("best.pt")

results = model.track(
    source="video.mp4",
    tracker="botsort_custom.yaml",
    conf=0.1,
    save=True,
    stream=True
)

for result in results:
    pass
```

botsort.yaml:

```yaml
tracker_type: botsort # (str) Tracker backend: botsort
track_high_thresh: 0.25 # (float) First-stage match threshold; raise for cleaner tracks, lower to keep more
track_low_thresh: 0.1 # (float) Second-stage threshold for low-score matches; balances recovery vs drift
```

```
new_track_thresh: 0.25 # (float) Start a new track if no match ≥ this; higher reduces false tracks
track_buffer: 30 # (int) Frames to keep lost tracks alive; higher handles occlusion, increases ID switches risk
match_thresh: 0.8 # (float) Association similarity threshold (IoU/cost); tune with detector quality
fuse_score: True # (bool) Fuse detection score with motion/IoU for matching; stabilizes weak detections


# BoT-SORT specifics
gmc_method: sparseOptFlow # (str) Global motion compensation: sparseOptFlow|orb|sift|ecc|none; helps moving camera scenes


# ReID model related thresh
proximity_thresh: 0.5 # (float) Min IoU to consider tracks proximate for ReID; higher is stricter
appearance_thresh: 0.8 # (float) Min appearance similarity for ReID; raise to avoid identity swaps
with_reid: False # (bool) Enable ReID model use; needs extra model and compute
model: auto # (str) ReID model name/path; "auto" uses detector features if available
```

botsort_custom.yaml:

```
tracker_type: botsort # (str) Tracker backend: botsort
track_high_thresh: 0.2 # (float) First-stage match threshold; raise for cleaner tracks, lower to keep more
track_low_thresh: 0.05 # (float) Second-stage threshold for low-score matches; balances recovery vs drift
new_track_thresh: 0.3 # (float) Start a new track if no match ≥ this; higher reduces false tracks
track_buffer: 90 # (int) Frames to keep lost tracks alive; higher handles occlusion, increases ID switches risk
match_thresh: 0.7 # (float) Association similarity threshold (IoU/cost); tune with detector quality
fuse_score: True # (bool) Fuse detection score with motion/IoU for matching; stabilizes weak detections


# BoT-SORT specifics
gmc_method: sparseOptFlow # (str) Global motion compensation: sparseOptFlow|orb|sift|ecc|none; helps moving camera scenes


# ReID model related thresh
proximity_thresh: 0.5 # (float) Min IoU to consider tracks proximate for ReID; higher is stricter
appearance_thresh: 0.8 # (float) Min appearance similarity for ReID; raise to avoid identity swaps
with_reid: False # (bool) Enable ReID model use; needs extra model and compute
model: auto # (str) ReID model name/path; "auto" uses detector features if available
```

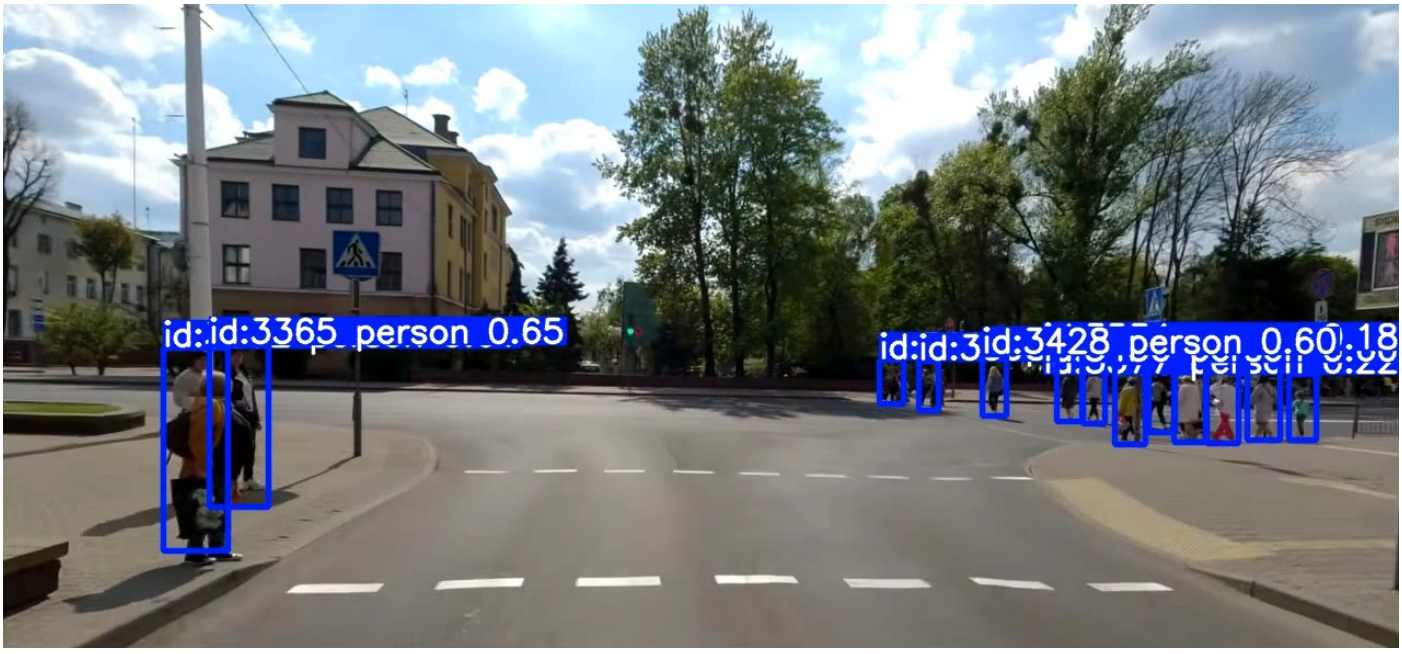
ByteTrack — алгоритм многократного сопровождения объектов, который сопоставляет детекции с существующими треками в два этапа, используя детекции с высокой и низкой уверенностью. Отличается высокой скоростью и небольшой вычислительной нагрузкой, но может терять идентификаторы при пересечении траекторий объектов.

BoT-SORT — алгоритм, который дополнительно поддерживает компенсацию движения камеры и повторную идентификацию объектов по внешнему виду (ReID). Это может повысить устойчивость отслеживания и уменьшить количество смен ID, но увеличить вычислительные затраты.

ByteTrack подходит для быстрого трекинга, а BoT-SORT предоставляет дополнительные возможности для сохранения идентификаторов в сложных сценах.

Результат



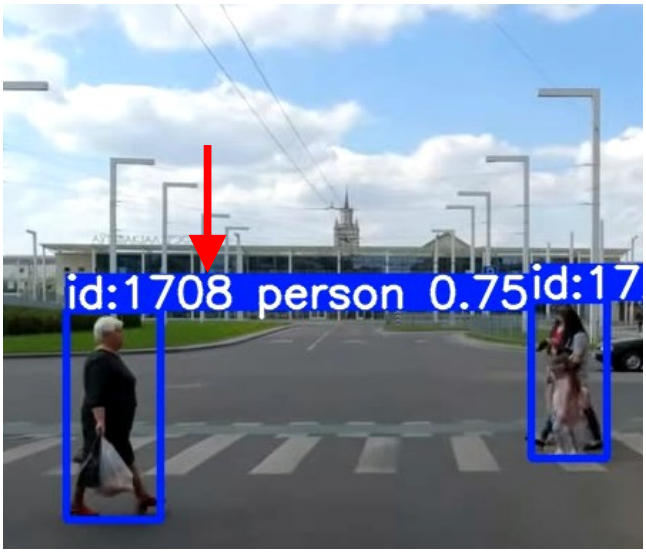


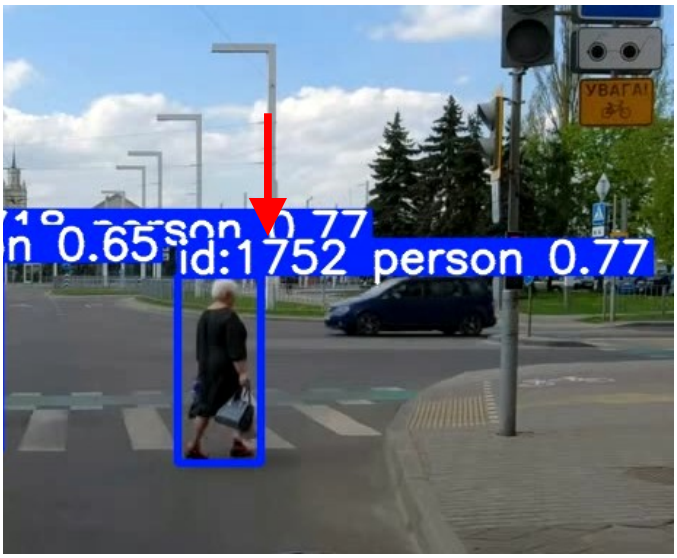
Исследовать изменения параметров в конфигурационных файлах и их влияние на качество трекинга;

Параметр	**Стандартный изменённый**	**Назначение и влияние**
track_high_thresh	0.25 - 0.2	Снижение порога позволяет учитывать больше детекций при сопоставлении треков.
track_low_thresh	0.1 - 0.05	Снижение порога помогает сохранять треки при слабых детекциях.
new_track_thresh	0.25 - 0.3	Повышение порога уменьшает вероятность

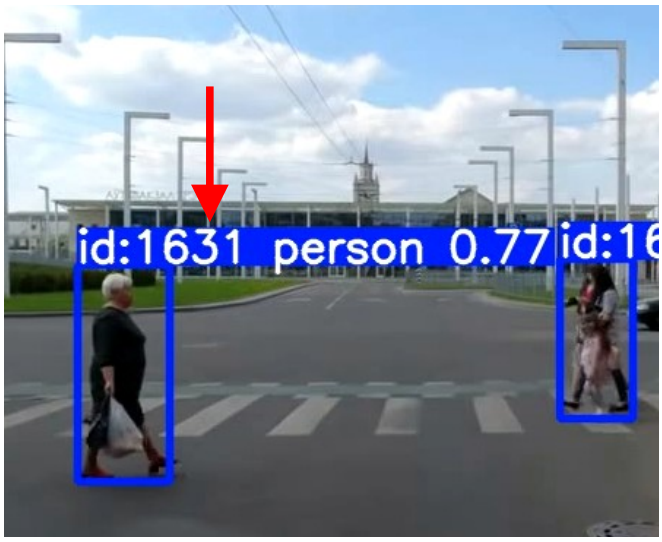
		создания новых ID для слабых детекций.
track_buffer	30 - 90	Увеличение времени хранения потерянного трека помогает восстановить прежний ID после временного пропадания человека.
match_thresh	0.8 - 0.7	Изменяет порог стоимости сопоставления треков и детекций; влияние зависит от сцены.

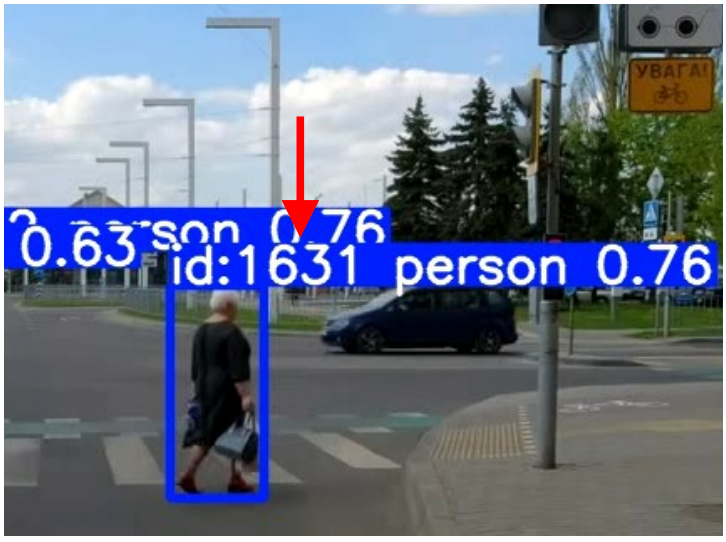
До:





После:





Улучшение стабильности ID связано прежде всего с увеличением track_buffer с 30 до 90 кадров, снижением track_high_thresh и track_low_thresh, а также повышением new_track_thresh. Это помогает сохранять существующие треки при временных пропусках детекций и уменьшает вероятность преждевременного создания нового ID.

Вывод: изучил применение алгоритмов трекинга на базе обученной сети-детектора объектов.